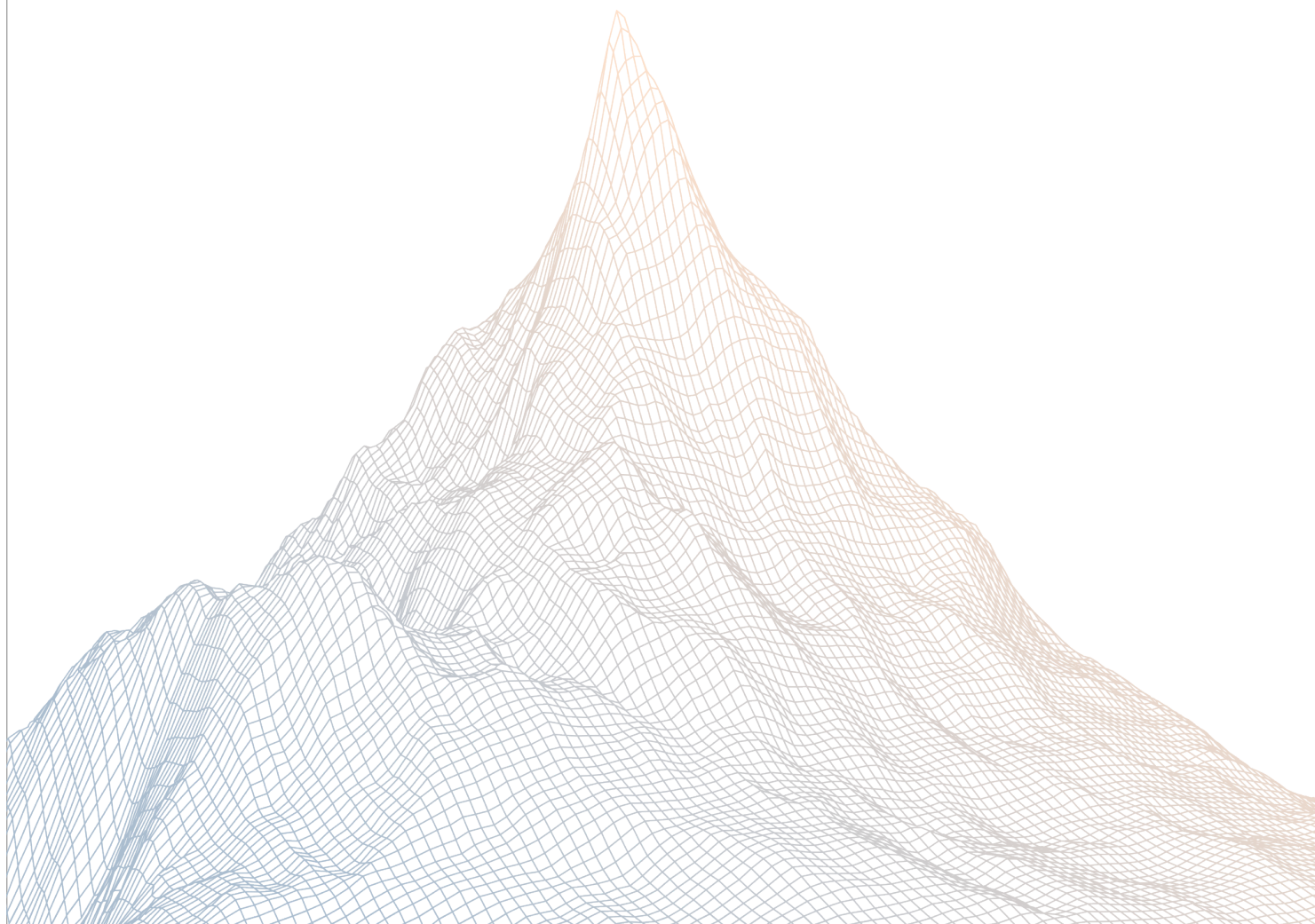


Supernova

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: January 16th to January 30th, 2026
AUDITED BY: hickup
said

Contents

1	Introduction	2
1.1	About Zenith	3
1.2	Disclaimer	3
1.3	Risk Classification	3
2	Executive Summary	3
2.1	About Supernova	4
2.2	Scope	4
2.3	Audit Timeline	5
2.4	Issues Found	5
2.5	Security note	5
3	Findings Summary	5
4	Findings	8
4.1	High Risk	9
4.2	Medium Risk	17
4.3	Low Risk	36
4.4	Informational	67

1

Introduction

1.1 About Zenith

Zenith assembles auditors with proven track records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at <https://zenith.security>.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Supernova

Supernova ParRate is an on-chain derivatives protocol that enables users to take fixed or floating exposure on yield-bearing debt instruments. The system creates markets that reference the price-per-share (PPS) growth of the underlying asset over a fixed maturity window, allowing traders to hedge or speculate on future borrowing costs, forward rates, or yield volatility.

By transforming PPS appreciation into a standardized settlement value, Supernova provides a transparent and composable way for expressing views on interest-rate movements across multiple chains.

2.2 Scope

The engagement involved a review of the following targets:

Target	superRateSwap
---------------	---------------

Repository	https://github.com/Supernova-Co/superRateSwap
-------------------	---

Commit Hash	eea37eb5e040586d78ee59655d78c5c03d48252c
--------------------	--

Files	RateSwapRouter.sol OrderBook/OrderBook.sol manager.sol OrderBook/OrderBookBase.sol vAMM/Vault.sol vAMM/vAMMSingleton.sol ClearingHouse.sol vAMM/RateSwapVammLib.sol AcrossBridge/SuperAcrossAdapter.sol epochReplay/CommitRevealAccumulator.sol epochReplay/GlobalSimpleOracle.sol epochReplay/EpochAaveAccumulator.sol libraries/IndexTickRays.sol SupernovaTransparentProxy.sol
--------------	--

2.3 Audit Timeline

January 16, 2026	Audit start
January 30, 2026	Audit end
February 5, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	3
Medium Risk	10
Low Risk	18
Informational	16
Total Issues	47

2.5 Security note

Considering the number of issues identified, it is statistically likely that there are more complex bugs still present that could not be identified given the time-boxed nature of this engagement. It is recommended that a follow-up audit and development of a more complex stateful test suite be undertaken prior to continuing to deploy significant monetary capital to production.

3

Findings Summary

ID	Description	Status
H-1	Funding continues to accrue post maturity	Resolved
H-2	AMM closePosition does not clear OrderBook orders	Resolved
H-3	Missing slippage protection for OrderBook swaps	Resolved
M-1	Arbitrary permit calldata can be used by relayers to drain funds from ClearingHouse / approvals given to Super-BridgeAdapter	Resolved
M-2	Position closures via orderbook may revert because solve-ForBaseNeededForQuote doesn't include fees	Resolved
M-3	setFundingFee doesn't sync funding first	Resolved
M-4	Self-foreclosure results in loss of funds from position deletion	Resolved
M-5	Missing slippage protection in closePosition	Resolved
M-6	Orderbook long-side matching can be DoSed with 0 makerBaseAtSpot	Resolved
M-7	foreclose leaves open orders and locks	Resolved
M-8	Just-in-time collateralization for ask limit orders will cause reverts upon fulfilment	Resolved
M-9	OrderBook DoS via dust orders	Resolved
M-10	Deposit to Vault at exact maturity skips syncFunding	Resolved
L-1	Order traversal can be DoSed	Resolved
L-2	quoteMarketSell missing fee deduction	Resolved
L-3	closePosition return incorrect values	Resolved
L-4	OrderBook settlement rounding allows zero quote trades	Resolved
L-5	Relayer can bypass router sessions via arbitrary oracleTarget.call	Resolved

ID	Description	Status
L-6	syncFunding() requires oracle data post maturity	Resolved
L-7	debitQuoteMarket returns incorrect flag when quoteAvail == amount	Resolved
L-8	Missing Slippage Protection in depositToVault/ withdrawFromVault	Acknowledged
L-9	Bad debt externalization via ClearingHouse.withdraw before the position is closed or liquidated	Acknowledged
L-10	Partial ClearingHouse credit not consumed before bad-debt absorption	Resolved
L-11	Incorrect start index calculation in initMarket	Resolved
L-12	Funding accrual underflow causes DoS	Resolved
L-13	Market creation is insufficiently robust	Resolved
L-14	Hardcoded maximum fee value is incorrect	Resolved
L-15	Lack of liquidator incentive leads to bad debt accumulation	Acknowledged
L-16	Solvency check division by zero	Resolved
L-17	Vault insolvency freezes market	Resolved
L-18	Missing deadline check in openSession	Resolved
I-1	openSession() can only be called by stored relayer	Acknowledged
I-2	Rounding dust from liquidation calculations	Resolved
I-3	p.base assignment can be refactored out	Resolved
I-4	ZeroOut condition can be simplified	Resolved
I-5	Break condition can be performed earlier	Resolved

ID	Description	Status
I-6	getLiquidationAPR() doesn't preview funding accrual	Resolved
I-7	Redundant re-computation of defaultMargin	Resolved
I-8	Redundant accrual preview in fetchPosition()	Resolved
I-9	ClearingHouse unit mismatch	Resolved
I-10	Typographical Errors	Resolved
I-11	Missing structs for typehashes	Resolved
I-12	USDC hardcoded value may have to be changed for MegaETH	Resolved
I-13	Struct memory copying is by reference	Resolved
I-14	Unsafe castings	Resolved
I-15	Incorrect natspec / comments	Resolved
I-16	Deposit can revert when vault headroom is zero	Resolved

4

Findings

4.1 High Risk

A total of 3 high risk findings were identified.

[H-1] Funding continues to accrue post maturity

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: High

Target:

- [Vault.sol#L110-L197](#)
- [Vault.sol#L363](#)
- [manager.sol#L214-L224](#)
- [manager.sol#L465-L493](#)

Description:

`settleFunding()` continues to update `accPerNotionalWad` for both long and short pools even after market maturity. This causes positions to continue accruing funding payments post-expiry when `withdrawFromMarket` or `withdrawFromExpiredMarket` is invoked, which should not happen.

`withdrawFromMarket()` / `withdrawFromExpiredMarket()` invokes `accrueToCollateral()`, which calls `manager.syncFunding()` → `vault.settleFunding()`, which updates `accPerNotionalWad` even past maturity and updates the position's base amount.

This means that positions continue to accrue funding based on post-maturity rate changes, which violates the expected behavior that funding should stop accruing once a market expires.

Furthermore, the vault's PnL is finalised upon the settlement at / past maturity, so any vault flows that occurred subsequently are not accounted for, resulting in potential loss of funds for outstanding LPs.

Recommendations:

Add an early return in `settleFunding` if the last settlement has been performed. Using `s.finalVaultAccrued` seems to be a good candidate as a flag for this, but it probably is safer to dedicate a boolean variable for it. It should also be used in `withdrawFromVault()` to

replace `finalVaultPnL` as this flag, because while unlikely, it could be the case that all trades were routed through the orderbook.

Supernova: Resolved with [PR# 131](#).

Zenith: Verified.

[H-2] AMM `closePosition` does not clear `OrderBook` orders

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [vAMMSingleton.sol#L272-L300](#)
- [manager.sol#L512-L530](#)

Description:

Closing a position via the AMM or liquidating a position does not cancel open orders or clear order locks. Those orders remain on the book but are no longer backed, and when matched upon `_settleFill`, subtraction underflow occurs when attempting to deduct from the user's base of quote and revert, blocking matching at that tick.

```
function _closePosition(
    bytes32 mktId,
    address user,
    Manager.Position memory p,
    bool isLiquidation
) internal returns (uint amountIn) {
    // return _closePosition(mktId, user, p, true);
    RateSwapVAMM.State memory m = markets[mktId];
    if (m.maturity <= block.timestamp) revert MktExpired();
    m = m.decayFixed(markets[mktId].price());
    uint256 baseStart = m.base;
    if (p.quote < 0) {
        uint baseIn = m.solveBaseInForQuoteOut(true, uint256(-p.quote));
        (RateSwapVAMM.State memory ns, uint quoteOut, )
        = m.swapBaseIn(baseIn); // Fees quote implicitly derived in vault
        p.quote += int256(quoteOut);
        if (p.base < baseIn) {
            uint deficit = baseIn - p.base;
            if (clearingHouse.userToAccountCredit(user) >= deficit) {
                manager.rawDebit(user, deficit);
                p.base += deficit;
                p.base -= baseIn;
            } else {
```

```

        manager. absorb(mktId, user, deficit);
>>>    p.base = 0;
    }
    } else {
>>>    p.base -= baseIn;
    }
    markets[mktId] = ns;
    amountIn = baseIn;
    _recordPrint(mktId, ns);
    emit SwapExecuted(mktId, user, true, amountIn, quoteOut);
} else if (p.quote > 0) {
    amountIn = uint(p.quote);
    (RateSwapVAMM.State memory ns, uint baseOut, uint feesBase)
= m.swapQuoteIn(
    uint256(p.quote)
);
>>>    p.quote = 0;
    p.base += baseOut;
    markets[mktId] = ns;
    _recordPrint(mktId, ns);
    emit SwapExecuted(mktId, user, false, amountIn, baseOut);
    if (feesBase > 0) manager. accrueFeesBaseToVault(mktId, feesBase);
}
int256 baseDiff = int(m.base) - int(baseStart);
manager. writeToPosition(mktId, user, p, baseDiff, isLiquidation);
emit PositionClosed(mktId, user);
}

```

```

function liquidate(bytes32 mktId, address user) external {
    accrueToCollateral(mktId, user);
    Position memory p = positions[mktId][user];
    bool _isSolvent = _checkSolvency(p, mktId, false);
    if (_isSolvent) revert NotUnderwater();
    uint baseIn = vamm. closeLiquidation(mktId, user, p);
    uint maxDue = (baseIn * 0.05e5) / 1e5;
    if (positions[mktId][user].base > maxDue) {
>>>    positions[mktId][user].base -= maxDue;
        clearingHouse. credit(msg.sender, (maxDue * 0.9e5) / 1e5);
        clearingHouse. credit(address(this), (maxDue * 0.1e5) / 1e5);
    } else {
        maxDue = positions[mktId][user].base;
>>>    positions[mktId][user].base = 0;
        clearingHouse. credit(msg.sender, (maxDue * 0.9e5) / 1e5);
        clearingHouse. credit(address(this), (maxDue * 0.1e5) / 1e5);
    }
}

```

```
emit PositionLiquidiated(mktId, user, msg.sender, baseIn, (maxDue
* 0.9e5) / 1e5);
}
```

```
function _settleFill(
    bytes32 mktId,
    address maker,
    address taker,
    Side takerSide,
    uint256 tradeSize,
    uint256 priceRay // 1e27
) internal {
    // _settleFill(mktId, maker.userToCredit, taker, Side.Long, tradeBase,
    indexRay);
    if (maker == taker) revert CannotFillSelf();
    // quote in 1e27 * base → scale back to 1e18 base units
    RateSwapVAMM.State memory m = vamm.marketsStruct(mktId);
    uint spotPrice = m.impliedSpot((priceRay - 1e27) / 1e9, block.timestamp);

    uint fees = uint(m.feeBps);
    Manager.Position memory pTaker = manager.fetchPosition(mktId, taker);
    Manager.Position memory pMaker = manager.fetchPosition(mktId, maker);

    uint256 quoteWei = (tradeSize * 1e18) / spotPrice;
    if (quoteWei > uint(type(int256).max)) revert TooMature();

    if (takerSide == Side.Short) {
        // Taker is selling quote → maker is the bidder (going long)
        // taker pays base output of trade to vault
        uint256 feeB = (tradeSize * fees) / 10_000;
        uint256 takerBaseNet = tradeSize - feeB;

        pTaker.base += uint256(takerBaseNet);
        pTaker.quote -= int256(quoteWei);

    >>> pMaker.base -= uint256(tradeSize);
        pMaker.quote += int256(quoteWei);
        if (feeB > 0) {
            manager accrueFeesBaseToVault(mktId, feeB);
            emit VaultOrderbookFeeAccrued(mktId, int(feeB));
        }
    } else {

        // Taker is buying selling → maker is the asker (going short)
        // taker pays quote output to vault
```

```
uint256 feeQ = (quoteWei * fees) / 10_000;
uint256 takerQuoteNet = quoteWei - feeQ;

pTaker.base -= uint256(tradeSize);
pTaker.quote += int256(takerQuoteNet);

pMaker.base += uint256(tradeSize);
pMaker.quote -= int256(quoteWei);
// Fee quote not explicitly needed as will be reflected in global diff
// in the vault contract
}

// Write to positions manager
manager.writeToPosition(mktId, taker, pTaker);
manager.writeToPosition(mktId, maker, pMaker);

emit Fill(mktId, maker, taker, takerSide, tradeSize, priceRay);
}
```

Recommendations:

Before closing using AMM and during liquidation, require the user to have no open orders, or to cancel and clear all orders and locks.

Supernova: Resolved with [PR #137](#).

Zenith: Verified.

[H-3] Missing slippage protection for OrderBook swaps

SEVERITY: High

IMPACT: High

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [RateSwapRouter.sol#L787-L831](#)

Description:

The RateSwapRouter._swap function ignores the minOut parameter for OrderBook execution, using hardcoded limits (type(uint256).max or 0).

```
function _swap(
    bytes32 mktId,
    bool onlyUseBook,
    bool onlyUseAmm,
    bool isLong,
    uint256 sizeIn,
    uint256 minOut,
    address _user
) internal returns (uint filled, uint256 out, uint256 effectivePriceRate) {
    if (onlyUseBook == onlyUseAmm) revert InvalidOption();
    if (onlyUseBook) {
        if (isLong) {
            > (filled, effectivePriceRate, out) = orderBook().marketBuy(
                OrderBook.MktBuyParams({
                    mktId: mktId,
                    sizeBase: sizeIn,
                    maxRateRay: type(uint256).max,
                    taker: _user,
                    bestAskRay: 0,
                    quoteDesired: 0
                })
            );
        } else {
            > (out, effectivePriceRate, filled) = orderBook().marketSell(
                OrderBook.MktSellParams({
                    mktId: mktId,
                    sizeQuote: sizeIn,
```

```
minRateRay: 0,
taker: _user,
bestBidRay: 0,
sizeBase: 0
})
);
}
}
if (onlyUseAmm) {
out = vammSingleton().swap(mktId, isLong, sizeIn, minOut, _user);
if (isLong) {
effectivePriceRate = (sizeIn * 1e27) / out;
} else {
effectivePriceRate = (out * 1e27) / sizeIn;
}
filled = sizeIn;
}
}
```

Recommendations:

Consider to check minOut against received amount.

Supernova: Resolved with [PR #122](#).

Zenith: Verified.

4.2 Medium Risk

A total of 10 medium risk findings were identified.

[M-1] Arbitrary permit calldata can be used by relayers to drain funds from ClearingHouse / approvals given to SuperBridgeAdapter

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [ClearingHouse.sol#L144](#)
- [SuperAcrossAdapter.sol#L69](#)

Description:

`ClearingHouse.depositWithPermit2()` allows a relayer-controlled flow to execute arbitrary calldata against the USDC token contract. Specifically, `approve()` can be invoked to grant any recipient allowance, which can then be abused to drain the entire USDC balance held by the ClearingHouse.

`SuperBridgeAdapter.bridgeFor()` likewise is susceptible, where `transferFrom()` can be invoked to drain any user approvals given to the adapter.

Recommendations:

While the function is named as `Permit2`, it's actually utilising the EIP-2612 permit. Instead of forwarding raw calldata, decode the permit parameters and call `permit` directly. Do likewise for the `SuperBridgeAdapter`.

Note that there are 2 permit functions: 1 that supports bytes memory signature, the other the v, r and s fields. The latter is shown below.

```
function depositWithPermit(  
    address user,  
    uint256 amount,  
    bytes calldata usdcPermitCalldata
```

```
) external whenNotPaused onlyRole(ROUTER_ROLE) {  
    // Decode to EIP-2612 permit fields  
    (address owner, address spender, uint256 value, uint256 deadline, uint8 v,  
     bytes32 r, bytes32 s) = abi  
    .decode(usdcPermitCalldata, (address, address, uint256, uint256, uint8,  
     bytes32, bytes32));  
  
    // Enforce invariants  
    if (owner != user) revert InvalidPermitOwner();  
    if (spender != address(this)) revert InvalidPermitSpender();  
  
    // Explicit EIP-2612 permit call  
    IUSDC(address(usdc)).permit(owner, spender, value, deadline, v, r, s);  
  
    uint256 usdcPrior = usdc.balanceOf(address(this));  
    usdc.transferFrom(user, address(this), amount);  
    uint256 usdcPost = usdc.balanceOf(address(this));  
    if (usdcPost - usdcPrior != amount) revert BadStateTransition();  
  
    _deposit(false, amount, user);  
    _debitGas(user);  
}
```

If permit2 is intended to be supported, then permitTransferFrom() needs to be invoked on the canonical Permit2 contract.

Supernova: Resolved with [PR# 148](#).

Zenith: Verified.

[M-2] Position closures via orderbook may revert because solveForBaseNeededForQuote doesn't include fees

SEVERITY: Medium

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [OrderBook.sol#L501](#)

Description:

The `solveForBaseNeededForQuote` view function calculates how much base is needed to acquire a target amount of quote. However, it doesn't account for fees charged during actual execution, causing it to underestimate the base required.

```
function solveForBaseNeededForQuote(
    bytes32 mktId,
    uint256 quoteWanted
) public view returns (uint256 filled, uint256 effectivePrice,
    uint256 quoteTraded) {
    // ...

    > uint256 takeBase = (takeQuote * spotPrice) / 1e18;

    toConsume -= takeQuote;
    quoteTraded += takeQuote;
    > filled += takeBase;
    remaining -= takeQuote;

    // ...
}
```

When `marketBuy` actually executes, fees are charged on the quote output.

```
function marketBuy(
    MktBuyParams memory params
)
public
onlyRole(ROUTER_ROLE)
```

```
returns (uint256 filled, uint256 effectivePrice, uint256 quoteTraded)
{
    // ...
    {
        uint256 qFill = (f * 1e18) / spotPrice;
        uint256 feeQ = (qFill * uint(m.feeBps)) / 10_000;
        uint qNet = qFill - feeQ;
        quoteTraded += qNet;
    }
    // ...
}
// ...
}
```

This function is used by OrderBook's `closePosition` to determine how much base to trade.

```
function closePosition(
    bytes32 mktId,
    address user
) external onlyRole(ROUTER_ROLE) returns (uint256 filled, uint256 ef,
    uint256 out) {
    // ...
    if (p.quote > 0) {
        // ..
    } else if (p.quote < 0) {
        (uint _base, , ) = solveForBaseNeededForQuote(mktId, uint(-p.quote));

        (uint _filled, uint _efPrice, uint _out) = marketBuy(
            OrderBook.MktBuyParams({
                mktId: mktId,
                sizeBase: uint(_base),
                maxRateRay: type(uint256).max,
                taker: user,
                bestAskRay: 0,
                quoteDesired: uint(-p.quote)
            })
        );
        // ...
    }
}
```

In the event where the user has sufficient `p.base`, there will be remaining `p.quote` that needs to be closed out. Otherwise, when there is a deficit, position closures will revert as `p.base` will be zero after the deficit is covered but zeroed subsequently, while there is negative `p.quote` remaining.

Recommendations:

Account for fees in the `solveForBaseNeededForQuote` by inflating the quote target.

Supernova: Resolved with [PR# 145](#).

Zenith: Verified.

[M-3] setFundingFee doesn't sync funding first

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [Vault.sol#L420-L426](#)

Description:

The setFundingFee function changes the funding fee rate without first syncing the outstanding funding.

```
function setFundingFee(bytes32 mktId, uint256 feeBps)
    external onlyRole(OPERATOR_ROLE) {
    if (feeBps > 10_000) revert FeeTooHigh();
    if (block.timestamp >= mkt[mktId].maturityTs) revert MarketMatured();
    uint oldFee = mkt[mktId].fundingFeeBps;
    mkt[mktId].fundingFeeBps = feeBps;
    emit FundingFeeSet(mktId, oldFee, feeBps);
}
```

When the fee is changed, any unsettled funding from the period since lastAccrued will be calculated using the new fundingFeeBps when settleFunding is next called. This causes retroactive misattribution of fees.

Recommendations:

Always sync funding before changing the fee rate.

Supernova: Resolved with [PR# 144](#).

Zenith: Verified.

[M-4] Self-foreclosure results in loss of funds from position deletion

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

See issue for details.

Description:

The `foreclose` function does not prevent a user from foreclosing their own position. When `user = msg.sender`, the position's quote and base values is added to itself, then zeroed when the position is deleted, resulting in a loss of funds.

Recommendations:

Add a check at the beginning of the function to prevent self-foreclosure:

```
if (user == msg.sender) revert CannotForecloseSelf();
```

Supernova: Resolved with [PR# 132](#).

Zenith: Verified.

[M-5] Missing slippage protection in `closePosition`

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [RateSwapRouter.sol#L226-L232](#)

Description:

The `closePosition` function allows users to close their entire position in a single transaction via `vamm` or `orderBook`.

```
function _closeOpenPosition(  
    bytes32 mktId,  
    bool onlyUseBook,  
    bool onlyUseAmm,  
    address user  
) internal returns (uint256 out) {  
    if (onlyUseBook == onlyUseAmm) revert InvalidOption();  
    if (onlyUseBook) {  
        (, , out) = orderBook().closePosition(mktId, user);  
    }  
    if (onlyUseAmm) {  
        out = vammSingleton().closeOpenPosition(mktId, user);  
    }  
}
```

This function does not accept a `minOut` (minimum assets received) or `maxRate` (worst acceptable price) parameter. It executes the trade at the current market price.

- AMM: If the pool is unbalanced or the user is sandwiched, they will receive significantly less than expected.
- OrderBook: It takes the best available orders. If liquidity is thin, it will sweep deep into the book, incurring high slippage.

Recommendations:

Add `minOut` parameter to `closePosition` and check against the received amount.

Supernova: Resolved with [PR #152](#).

Zenith: Verified.

[M-6] Orderbook long-side matching can be DoSed with 0 makerBaseAtSpot

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [manager.sol#L336-L342](#)

Description:

In `_matchAtTickLong`, if `makerBaseAtSpot` rounds to zero, the loop advances head without reducing `mq.totalSize` or `b.sizeAsk`. This can leave stale liquidity at the best ask and cause `marketBuy` to OOG.

```
function _matchAtTickLong(
    Book storage b,
    bytes32 mktId,
    uint256 tick,
    uint256 takerSize, // BASE
    address taker
) private returns (uint256 filled, uint256 indexRay) {
    // ...

    while (takerSize > 0 && head < mq.orders.length) {
        Order storage maker = mq.orders[head];

        {
            uint256 makerQuote = maker.size; // QUOTE
            uint256 makerBaseAtSpot = (makerQuote * spotPrice) / 1e18;
            uint256 tradeBase = makerBaseAtSpot < takerSize ? makerBaseAtSpot :
                takerSize;
            if (tradeBase == 0) {
                head++;
            } else {
                uint256 quoteUsed = (tradeBase * 1e18) / spotPrice;
                if (quoteUsed > makerQuote) {
                    quoteUsed = makerQuote;
                }
            }
        }
    }
}
```

```
// update maker (quote)
maker.size = makerQuote - quoteUsed;
mq.totalSize -= quoteUsed;
sizeAsk -= quoteUsed; // ← update local, not storage

// update taker (base)
takerSize -= tradeBase;
filled += tradeBase;

_settleFill(mktId, maker.userToCredit, taker, Side.Long, tradeBase,
            indexRay);

if (maker.size == 0) {
  _onOrderFullyFilled(mktId, maker.userToCredit, maker.orderId);
  head++;
}
}
}
}

// ...
}
```

Recommendations:

Treat dust orders as fully filled.

Supernova: Resolved with [PR #134](#) and [PR #153](#)

Zenith: Verified.

[M-7] foreclose leaves open orders and locks

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L538-L562](#)

Description:

foreclose deletes the user's position but does not cancel open orders or clear locks.

```
function foreclose(bytes32 mktId, address user, uint256 capitalInfusion)
    external {
    if (!isForeclosureAllowed) revert NotEnabled();
    accrueToCollateral(mktId, user);
    accrueToCollateral(mktId, msg.sender);

    Position memory toTakeOver = positions[mktId][user];
    Position memory toGive = positions[mktId][msg.sender];

    /// user must be insolvent at least stringent check
    if (!_checkSolvency(toTakeOver, mktId, false)) revert NotUnderwater();

    clearingHouse.debit(msg.sender, capitalInfusion);
    toGive.base += capitalInfusion;

    toGive.quote += toTakeOver.quote;
    toGive.base += toTakeOver.base;

    /// User taking over the position must be solvent at most stringent check
    if (!_checkSolvency(toGive, mktId, true)) {
    revert SolvencyCheckFailed();
    }
    emit UserForeclosed(mktId, user, msg.sender, toTakeOver);
    positions[mktId][msg.sender] = toGive;
    delete positions[mktId][user];
    }
```

Solvency is enforced inside `Manager.writeToPosition` and is called during orderbook matching, `OrderBookBase._settleFill` calls `manager.writeToPosition(mktId, taker, pTaker)` and `manager.writeToPosition(mktId, maker, pMaker)`. So when a foreclosed user's resting order is matched, `_settleFill` tries to write the maker's position and hits `_checkSolvency`, which can revert if their collateral was deleted.

Besides that, clear order locks also not possible due to empty positions.

```
function clearOrderLock(bytes32 mktId, address user, uint256 orderId)
    external onlyRole(MARKET_ROLE) {
    OrderLocks memory l = orderLocks[mktId][orderId];
    if (l.baseLocked > 0 || l.quoteLocked > 0) {
    Position storage p = positions[mktId][user];

    if (l.baseLocked > 0) {
    p.baseLockedForOrders -= l.baseLocked;
    }
    if (l.quoteLocked > 0) {
    p.quoteLockedForOrders -= uint256(l.quoteLocked);
    }

    delete orderLocks[mktId][orderId];
    }
}
```

Recommendations:

Require no open orders before `foreclose` or `cancel` orders and clear locks during `foreclose`.

Supernova: Resolved with [PR #137](#).

Zenith: Verified.

[M-8] Just-in-time collateralization for ask limit orders will cause reverts upon fulfilment

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L325-L350](#)

Description:

debitQuoteMarket fails to account for the LTV and safe check that is performed by the solvency check when the order gets filled, resulting in DoS.

```
function debitQuoteMarket(
    bytes32 mkt,
    address user,
    uint256 amount
    // bool hold
) external onlyRole(MARKET_ROLE) returns (Position memory p,
    uint256 defaultMargin) {
    accrueToCollateral(mkt, user);
    p = positions[mkt][user];
    defaultMargin = vamm.solveForBaseNeededForQuote(mkt, amount);
    int quoteAvail = (p.quote - int(p.quoteLockedForOrders));
    uint baseAvail = (p.base - p.baseLockedForOrders);
    if (quoteAvail < int(amount) && defaultMargin > baseAvail) {
        // Ensure collateral is posted such that it covers initial cost of short
        defaultMargin = vamm.solveForBaseNeededForQuote(mkt, amount);
        clearingHouse.debit(user, defaultMargin);
        p.base += defaultMargin;
        positions[mkt][user] = p;
    }
    // Return flag used to lock base or quote in Orderbook
    if (quoteAvail > int(amount)) {
        defaultMargin = 0;
    }

    return (p, defaultMargin);
}
```

The subsequent call to `manager.writeToPosition` enforces a strict solvency check based on the market's LTV. if `collateral < required`, the transaction reverts.

```
function _checkSolvency(
    Position memory p,
    bytes32 mktId,
    bool isSafeCheck
) internal view returns (bool isSolvent) {
    uint256 price = vamm.getTimeWeightedPricePerMarketCurrent(mktId);
    uint256 ltvForMarket = mktIdToLtv[mktId];
    // Lower the LTV by 10% to reduce liquidation stress, if applicable
    if (isSafeCheck) ltvForMarket = (ltvForMarket * 90) / 100;
    if (p.quote >= 0) {
        // Margin For Long, not allowed
        isSolvent = true;
    } else {
        uint variableLegDebt = (uint(-p.quote) * price) / 1e18;
        uint health = (variableLegDebt * 1e5) / p.base; // debt // collateral
        isSolvent = health <= ltvForMarket;
    }
}
```

Recommendations:

Calculate required margin based on LTV + buffer in `debitQuoteMarket`.

```
if (quoteAvail < int(amount) && defaultMargin > baseAvail) {
    // Ensure collateral is posted such that it covers initial cost of short
    uint256 ltvForMarket = (90 * mktIdToLtv[mkt]) / 100;
    defaultMargin = (defaultMargin * 1e5) / ltvForMarket;
```

Supernova: Resolved with [PR #136](#).

Zenith: Verified.

[M-9] OrderBook DoS via dust orders

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [OrderBookBase.sol#L382-L441](#)

Description:

The OrderBook allows users to place orders with a size of 1 wei (dusts). A malicious user can place thousands of dust orders to grief takers with block gas limit DoS.

```
function placeLimit(
    Book storage b,
    Side side,
    bytes32 mktId,
    address user,
    uint256 limitIndexRay,
    uint256 size
) internal returns (uint256 orderId, uint256 tick) {
    if (size == 0) revert SizeEq0();
    // snap limit to a valid tick price (optional but safer)
    uint256 snapped = snapRate(b, limitIndexRay);
    if (snapped == 1e27) revert TickZero();
    tick = indexToTick(b, snapped);

    // bump id
    orderId = ++b.nextOrderId;

    // choose queue
    Queue storage q = (side == Side.Long) ? b.long[tick] : b.short[tick];

    if (side == Side.Long) {
        b.sizeBid += size;
    } else {
        b.sizeAsk += size;
    }

    // append
```



```
q.orders.push(  
  Order({  
    mktId: mktId,  
    userToCredit: user,  
    limitIndexRay: snapped,  
    size: size,  
    orderId: orderId  
  })  
);  
q.totalSize += size;  
  
// save location for cancels  
b.loc[orderId] = Location({  
  exists: true,  
  side: side,  
  tick: tick,  
  index: q.orders.length - 1  
});  
  
// maintain top-of-book  
if (side == Side.Long) {  
  if (q.totalSize == size && (tick > b.bestBidTick)) {  
    b.bestBidTick = tick;  
  }  
} else {  
  if (q.totalSize == size && (tick < b.bestAskTick)) {  
    b.bestAskTick = tick;  
  }  
}  
  
emit OrderPlaced(mktId, user, side, orderId, tick, snapped, size);  
}
```

Recommendations:

Enforce a minimum order size.

Supernova: Resolved with [PR #123](#).

Zenith: Verified.

[M-10] Deposit to Vault at exact maturity skips syncFunding

SEVERITY: Medium

IMPACT: Medium

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [Vault.sol#L335](#)

Description:

In `depositToVault`, the logic to sync funding uses strict inequality.

```
function depositToVault(
    bytes32 mktId,
    address user,
    uint256 amountAsset
) external onlyRole(ROUTER_ROLE) returns (uint256 shares) {
    Mkt storage s = mkt[mktId];
    if (isSubscriptionClosed[mktId]) revert SubscriptionClosed();
    > if (s.maturityTs > block.timestamp) manager.syncFunding(mktId);
    if (block.timestamp > s.maturityTs) revert SubscriptionClosed();
    if (amountAsset + s.totalAssets > s.vaultCap) revert VaultCapExceeded();
    clearingHouse.debit(user, amountAsset);
    if (s.totalShares != 0) {
        shares = (amountAsset * s.totalShares)
            / uint(_vaultHeadroomLessAccrued(mktId));
    } else {
        shares = amountAsset;
    }
    balance[mktId][user] += shares;
    s.totalShares += shares;
    s.totalAssets += amountAsset;
    emit DepositToVaultMarket(mktId, user, amountAsset, shares);
}
```

If `block.timestamp == s.maturityTs`, this check fails, and `syncFunding` is skipped. However, the subsequent check `if (block.timestamp > s.maturityTs) revert` also pass, allowing execution to proceed. This allows a user to deposit exactly at the maturity timestamp using stale vault PnL data (missing the final funding accrual). This can be

exploited to mint shares at an incorrect price.

Recommendations:

Prevent deposits at maturity.

```
if (block.timestamp > s.maturityTs) revert SusbcriptionClosed();  
if (block.timestamp ≥ s.maturityTs) revert SusbcriptionClosed();
```

Supernova: Resolved with [PR #118](#).

Zenith: Verified.

4.3 Low Risk

A total of 18 low risk findings were identified.

[L-1] Order traversal can be DoSed

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [OrderBook.sol#L96](#)
- [OrderBook.sol#L123](#)
- [OrderBook.sol#L139](#)
- [OrderBook.sol#L159](#)
- [OrderBook.sol#L168-L170](#)
- [OrderBook.sol#L178-L198](#)
- [OrderBookBase.sol#L316-L335](#)
- [OrderBook.sol#L683-L722](#)

Description:

- [On MegaEth](#), accessing block environment data like `block.timestamp` caps the remaining compute gas to 20M.
- The `OrderBook` maintains a mapping `userOrders[mktId][user]` that stores an array of order IDs for each user per market. There is no limit on the number of orders a user can place, and several operations iterate through this array linearly, creating a DoS risk via block gas limit exhaustion.
- In a similar fashion, one can repeatedly create and cancel orders to force the queue to grow large enough that iterating through these cancelled orders can revert from exceeding the gas limit. This is especially so for `_matchAtTickShort()` which doesn't skip past zero size orders.

Recommendations:

- Replace the linear search in `_removeUserOrder` with a reverse index mapping (`userOrderIndex`) to make removal $O(1)$ instead of $O(n)$.

- `_matchAtTickShort()`, like `_matchAtTickLong()`, should have a short-circuit for zero size orders.
- Add a function to manually move `q.head` past cancelled orders.

```
function skipZeroSizeOrders(  
  Book storage b,  
  bytes32 mktId,  
  Side side,  
  uint256 tick,  
  uint256 endIndex  
) internal {  
  Queue storage q = (side == Side.Long) ? b.long[tick] : b.short[tick];  
  uint256 head = q.head;  
  uint256 maxIdx = endIndex < q.orders.length ? endIndex : q.orders.length;  
  while (head < maxIdx && q.orders[head].size == 0) {  
    head++;  
  }  
}
```

Supernova: Resolved with [PR# 149](#).

Zenith: Verified.

[L-2] quoteMarketSell missing fee deduction

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [OrderBook.sol#L585-L586](#)

Description:

The quoteMarketSell view function is intended to mirror the execution logic of marketSell for quoting purposes. However, it fails to deduct fees from the filled amount, causing it to return higher values than actual execution produces.

In marketSell, fees are correctly deducted:

```
function marketSell(
    MktSellParams memory params
) public onlyRole(ROUTER_ROLE) returns (uint256 filled,
    uint256 effectivePrice, uint256 quoteTraded) {
    // ...

    // SHORT: taker sells quote, receives base → fee on base (output-side)
    {
        uint256 feeB = (fBase * uint(m.feeBps)) / 10_000;
        uint256 fBaseNet = fBase - feeB;
        filled += fBaseNet;
    }
    quoteTraded += (fBase * 1e18) / spotPrice;
    // ...
}
```

But in quoteMarketSell, fees are not deducted, causing view function returns higher filled amount than actual execution:

```
function quoteMarketSell(
    bytes32 mktId,
    uint256 sizeQuote,
    uint256 minRateRay
```

```
) external view returns (uint256 filled, uint256 effectivePrice,  
    uint256 quoteTraded) {  
    // ...  
  
    filledBase += fBase;  
    quoteTraded += quoteConsumed;  
    remainingQuote -= quoteConsumed;  
    toConsume -= fBase;  
    // ...  
}
```

Recommendations:

Add fee deduction to match execution logic.

Supernova: Resolved with [PR# 146](#).

Zenith: Verified.

[L-3] closePosition return incorrect values

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [OrderBook.sol#L223-L225](#)
- [OrderBook.sol#L223-L225](#)

Description:

The closePosition function has multiple issues with its return values.

1. Swapped return values when p.quote > 0 :

```
▷ /// @return filled The amount of base filled  
/// @return ef The effective price of the close  
▷ /// @return out The amount of quote filled  
function closePosition(  
bytes32 mktId,  
address user  
) external onlyRole(ROUTER_ROLE) returns (uint256 filled, uint256 ef,  
uint256 out) {  
Manager.Position memory p = manager.fetchPosition(mktId, user);  
if (p.quote > 0) {  
(uint _filled, uint _efPrice, uint _out) = marketSell(  
OrderBook.MktSellParams({  
mktId: mktId,  
sizeQuote: uint(p.quote),  
minRateRay: 0,  
taker: user,  
bestBidRay: 0,  
sizeBase: 0  
})  
);  
▷ out = _filled; // base filled from marketSell, but put it as quote filled  
▷ filled = _out; // quoteTraded from marketSell, but put it as base filled  
ef = _efPrice;  
} else if (p.quote < 0) {
```



```
// ...  
}  
}
```

The function signature declares returns (uint256 filled, uint256 ef, uint256 out) where filled should be base and out should be quote, but the assignments are reversed.

```
function marketSell(  
  MktSellParams memory params  
)  
public  
onlyRole(ROUTER_ROLE)  
> returns (uint256 filled, uint256 effectivePrice, uint256 quoteTraded)  
// ...
```

2. Missing return values when p.quote < 0, When closing a short position, filled and ef are never set, returning incorrect zeros.

```
// ...  
} else if (p.quote < 0) {  
  (uint _base, , ) = solveForBaseNeededForQuote(mktId, uint(-p.quote));  
  (uint _filled, uint _efPrice, uint _out) = marketBuy(...);  
  out = _out;  
  // filled and ef are NOT assigned! They remain 0  
}  
// ...
```

Recommendations:

Fix the return value assignments.

Supernova: Resolved with [PR #152](#).

Zenith: Verified.

[L-4] OrderBook settlement rounding allows zero quote trades

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [OrderBookBase.sol#L129-L130](#)

Description:

The `_settleFill` function converts base trade size to quote using round down division. When `spotPrice > 1e18`, a trade with `tradeSize` could produce `quoteWei = 0`.

```
function _settleFill(
    bytes32 mktId,
    address maker,
    address taker,
    Side takerSide,
    uint256 tradeSize,
    uint256 priceRay // 1e27
) internal {
    // ...
    > uint256 quoteWei = (tradeSize * 1e18) / spotPrice;
    if (quoteWei > uint(type(uint256).max)) revert TooMature();
    if (takerSide == Side.Short) {
        // Taker is selling quote → maker is the bidder (going long)
        // taker pays base output of trade to vault
        uint256 feeB = (tradeSize * fees) / 10_000;
        uint256 takerBaseNet = tradeSize - feeB;

        > pTaker.base += uint256(takerBaseNet);
        > pTaker.quote -= int256(quoteWei);

        > pMaker.base -= uint256(tradeSize);
        > pMaker.quote += int256(quoteWei);
        if (feeB > 0) {
            manager accrueFeesBaseToVault(mktId, feeB);
            emit VaultOrderbookFeeAccrued(mktId, int(feeB));
        }
    } else {
```

```
// ...
uint256 feeQ = (quoteWei * fees) / 10_000;
uint256 takerQuoteNet = quoteWei - feeQ;

pTaker.base -= uint256(tradeSize);
pTaker.quote += int256(takerQuoteNet);

pMaker.base += uint256(tradeSize);
pMaker.quote -= int256(quoteWei);
// Fee quote not explicitly needed as will be reflected in global diff
// in the vault contract
}
// ...
}
```

When `quoteWei = 0`, the settlement proceeds. For a `Side.Short` taker matching against a bid. The taker receives base for free, and the maker loses base with no quote compensation.

For instance, if price is $1.1e18$, user can call `marketSell` with `sizeQuote` resulting in `sizeBase = 1`, then `_matchAtTickShort` sets `tradeBase = 1` and `_settleFill` computes $quoteWei = (1 * 1e18) / 1.1e18 = 0$.

Recommendations:

Consider to round-up `quoteWei` calculation when `takerSide` is `Side.Short`.

Supernova: Resolved with [PR# 143](#).

Zenith: Verified.

[L-5] Relay can bypass router sessions via arbitrary `oracleTarget.call`

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [RateSwapRouter.sol#L728](#)

Description:

`pushOracleAndBatch` allows relay to call any address via `oracleTarget.call(data)` before executing the router batch. There is no allowlist or selector check. Since the call is made from the Router itself, `msg.sender` for the target is the router.

```
function pushOracleAndBatch(
    bytes[] calldata calls,
    address oracleTarget,
    bytes calldata data
) external onlyRole(RELAYER_ROLE) returns (bytes[] memory results) {
    (bool ok, bytes memory err) = oracleTarget.call(data);
    if (!ok) {
        // bubble revert
        if (err.length > 0) {
            assembly {
                revert(add(err, 0x20), mload(err))
            }
        } else {
            revert BatchCallFailed();
        }
    }
    return _batchExecute(calls);
}
```

This enables a relay to directly invoke functions on contracts that trust the Router via `onlyRole(ROUTER_ROLE)` (e.g., Vault, Manager, OrderBook, vAMM), bypassing session/signature restrictions enforced in the Router.

Recommendations:

Restrict `oracleTarget` to a fixed oracle address or allowlist.

Supernova: Resolved with [PR# 139](#).

Zenith: Verified.

[L-6] syncFunding() requires oracle data post maturity

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L450-L457](#)

Description:

syncFunding() passes vault.lastAccrued(mktId) and block.timestamp as the start and end times when querying the oracle for growth. This creates an unnecessary dependency on post-maturity oracle data for operations that invoke syncFunding after a market has expired (e.g. withdrawFromExpiredMarket, accrueToCollateral). If the oracle has not been updated with post-maturity data, or does not support/expect queries beyond maturity, these transactions may fail.

Recommendations:

Cap the oracle query start and end times to market maturity.

```
function syncFunding(bytes32 mktId) public {
    uint256 maturityTs = marketToGlobalState[mktId].maturityTs;

    // startTs = min(vault.lastAccrued(mktId), maturityTs)
    uint256 startTs = vault.lastAccrued(mktId);
    if (maturityTs < startTs) startTs = maturityTs;

    // endTs = min(block.timestamp, maturityTs)
    uint256 endTs = block.timestamp;
    if (maturityTs < endTs) endTs = maturityTs;
    uint256 _growthRay
        = globalOracle.getGrowthFloat(marketToGlobalState[mktId].underlying,
            startTs, endTs);
    vault.settleFunding(mktId, _growthRay);
}
```

Supernova: Resolved with [PR# 130](#).

Zenith: Verified.

[L-7] debitQuoteMarket returns incorrect flag when quoteAvail == amount

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L344-L347](#)

Description:

When the user has exactly the quote needed for an ask such that `quoteAvail = amount`, `debitQuoteMarket` does not set `defaultMargin = 0`. This means that the default margin returned is non-zero, which results in the base amount being locked instead of the quote.

This would be an inconvenience for subsequent actions, like attempting to perform other actions subsequently that utilise `debitQuoteMarket()` because it'll revert from arithmetic underflow when calculating `baseAvail = p.base - p.baseLockedForOrders`.

Recommendations:

```
// Return flag used to lock base or quote in Orderbook
if (quoteAvail > int(amount)) {
  if (quoteAvail ≥ int(amount)) {
    defaultMargin = 0;
  }
}
```

Supernova: Resolved with [PR #128](#).

Zenith: Verified.

[L-8] Missing Slippage Protection in depositToVault/withdrawFromVault

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [RateSwapRouter.sol#L862-L876](#)

Description:

The RateSwapRouter functions for Vault interaction (depositToVault and withdrawFromVault) do not accept slippage protection parameters.

```
function depositToVault(bytes32 mktId, uint256 amount)
    external isAllowed returns (uint256 shares) {
    shares = _depositToVault(mktId, msg.sender, amount);
}

function withdrawFromVault(bytes32 mktId, uint256 shares)
    external isAllowed returns (uint256 amount) {
    amount = _withdrawFromVault(mktId, msg.sender, shares);
}
```

In the vault, the share price is determined by totalAssets + PnL. This value can fluctuate significantly based on market activity (funding payments) or potentially manipulation. Deposit without minShares, a user can receive fewer shares than expected. Similarly, withdraw without minAssets, a user can be forced to exit at a temporary price lower than expected.

Recommendations:

Consider to update depositToVault to accept minShares and withdrawFromVault to accept minAssets.

Supernova: Acknowledged. Once the PPS increases it will remain constant across the epoch.

[L-9] Bad debt externalization via `ClearingHouse.withdraw` before the position is closed or liquidated

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [vAMMSingleton.sol#L274-L281](#)

Description:

Users can withdraw their `ClearingHouse` credit before calling `closePosition` on a losing short or getting liquidated.

```
function _withdraw(uint256 _amount, address _user) internal {
    ClearingHouseStorage storage $ = _getClearingHouseStorage();
    uint gasCredited;
    $.userToAccountCredit[_user] -= _amount;
    if ($.userToAccountCredit[_user] == 0) {
        gasCredited = $.userToGasBalance[_user];
        _amount += gasCredited;
        emit AccountGasDebited(true, _user, gasCredited);
        $.userToGasBalance[_user] = 0;
    }
    usdc.transfer(_user, _amount / 1e12);
    emit AccountDebited(true, _user, _amount);
}
```

In `_closePosition`, if the user's base is insufficient, the contract attempts to debit the deficit from `ClearingHouse.userToAccountCredit`. If that balance is empty, the deficit is instead absorbed by the vault via `manager.absorb`, socializing the loss to vault.

```
function _closePosition(
    bytes32 mktId,
    address user,
    Manager.Position memory p,
    bool isLiquidation
) internal returns (uint amountIn) {
    // return _closePosition(mktId, user, p, true);
}
```

```

RateSwapVAMM.State memory m = markets[mktId];
if (m.maturity <= block.timestamp) revert MktExpired();
m = m.decayFixed(markets[mktId].price());
uint256 baseStart = m.base;
if (p.quote < 0) {
    uint baseIn = m.solveBaseInForQuoteOut(true, uint256(-p.quote));
    (RateSwapVAMM.State memory ns, uint quoteOut, )
    = m.swapBaseIn(baseIn); // Fees quote implicitly derived in vault
    p.quote += int256(quoteOut);
    if (p.base < baseIn) {
        uint deficit = baseIn - p.base;
        > if (clearingHouse.userToAccountCredit(user) >= deficit) {
            manager.rawDebit(user, deficit);
            p.base += deficit;
            p.base -= baseIn;
        } else {
            > manager.absorb(mktId, user, deficit);
            p.base = 0;
        }
    } else {
        p.base -= baseIn;
    }
    // ...
}

```

This allows a user to preserve funds while forcing bad debt onto the vault.

Recommendations:

Consider to prevent `ClearingHouse.withdraw` that while position user have insolvent position.

Supernova: Acknowledged. CH balances do not add to a users solvency and consequently should not need to be clawed from given efficient liquidations.

Zenith:

[L-10] Partial ClearingHouse credit not consumed before bad-debt absorption

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [vAMMSingleton.sol#L274-L281](#)

Description:

When closing a short position, if the user's base is insufficient, the code checks whether the entire deficit can be covered by `ClearingHouse.userToAccountCredit`. If not, it ignores any partial credit and absorbs the full deficit into the vault. This allows users with some remaining credit to avoid paying it, shifting more bad debt to vault.

```
function _closePosition(
    bytes32 mktId,
    address user,
    Manager.Position memory p,
    bool isLiquidation
) internal returns (uint amountIn) {
    // ...
    if (p.quote < 0) {
        uint baseIn = m.solveBaseInForQuoteOut(true, uint256(-p.quote));
        (RateSwapVAMM.State memory ns, uint quoteOut, )
        = m.swapBaseIn(baseIn); // Fees quote implicitly derived in vault
        p.quote += int256(quoteOut);
        if (p.base < baseIn) {
            uint deficit = baseIn - p.base;
            // @audit - partial rawDebit should be performed
            if (clearingHouse.userToAccountCredit(user) >= deficit) {
                manager.rawDebit(user, deficit);
                p.base += deficit;
                p.base -= baseIn;
            } else {
                manager.absorb(mktId, user, deficit);
                p.base = 0;
            }
        } else {
```

```
p.base -= baseIn;  
}  
// ...  
}
```

Recommendations:

```
} else {  
    uint256 userCredit = clearingHouse.userToAccountCredit(user);  
    manager.rawDebit(userCredit);  
    deficit -= userCredit;  
    manager.absorb(mktId, user, deficit);  
    p.base = 0;  
}
```

Supernova: Resolved with [PR #125](#).

Zenith: Verified.

[L-11] Incorrect start index calculation in initMarket

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L100](#)

Description:

The `initMarket` function accepts a `_startTs` parameter, which normally defines the scheduled start of the market duration. However, when fetching the initial `startRayIndex`, it uses `block.timestamp` instead of `_startTs`:

```
function initMarket(
    bytes32 _mktId,
    address _underlying,
    uint256 _startTs,
    uint256 _maturity,
    uint256 _ltvForMarket
) public onlyRole(MARKET_ROLE) returns (uint256 startRayIndex) {
    startRayIndex = globalOracle.getIndexAtTimestampForMarket(
        _underlying,
        block.timestamp
    );
    GlobalMarketState memory g = GlobalMarketState({
        underlying: _underlying,
        startRay: startRayIndex,
        maturityTs: _maturity
    });
    marketToGlobalState[_mktId] = g;
    vault.initializeMarket(_mktId, _startTs, _maturity);
    book.createBook(
        _mktId,
        _maturity,
        // 1e27, // The minimum rate acceptable by book, cannot go below current
        // price
        0.001e27
    );
    mktIdToLtv[_mktId] = _ltvForMarket;
```

```
}
```

If `startRayIndex` using `now`, but the true index at `_startTs` will be higher (assuming monotonic growth). That means once the market starts, pricing/interest calculations are anchored to a too-low start index.

If the market is initialized after `_startTs`, the `startRayIndex` will be higher than the actual index at `_startTs`. This effectively erases the interest accrued between `_startTs` and `block.timestamp` from the market's lifecycle. Since `_startTs` is passed to `vault.initializeMarket` and used for maturity calculations, the system has a mismatched state.

Recommendations:

Use `_startTs` to fetch the index or ensure `_startTs = block.timestamp`.

Supernova: Resolved with [PR #124](#)

Zenith: Verified.

[L-12] Funding accrual underflow causes DoS

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L485](#)

Description:

In `accrueToCollateral`, the logic handles short funding payments (short pays long) as:

```
function accrueToCollateral(  
    bytes32 mktId,  
    address user  
) public returns (int256 claimWad) {  
    // ...  
} else {  
    claimWad = -int(payment);  
    p.base -= uint(-claimWad);  
}  
// ...  
}
```

`p.base` is `uint256`. If `uint(-claimWad)` is greater than `p.base`, this operation will revert due to underflow.

`accrueToCollateral` is called at the beginning of every state-changing function (`liquidate`, `closePosition`, `swap`). If a user's funding debt exceeds their collateral, they are insolvent and should be liquidated. However, `liquidate` calls `accrueToCollateral` and reverts.

Recommendations:

Implement bad debt handling in `accrueToCollateral`.

Supernova: Resolved with [PR #151](#).

Zenith: Verified.

[L-13] Market creation is insufficiently robust

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [vAMMSingleton.sol#L110-L120](#)
- [manager.sol#L91-L97](#)
- [Vault.sol#L85-L89](#)

Description:

While market creation is permissioned, there are a number of issues pertaining to it that potentially harms existing markets:

1. Quote, time and fee parameters aren't sanity checked.
 - `tokenBInitial = markets[key].quote` must be non-zero to prevent `state.price()` from reverting from division by zero.
 - The start and maturity could benefit from ensuring that `block.timestamp ≤ start ≤ maturity`
 - fee is checked to be at most 50%, but isn't done so at market creation.
2. The market ID derivation does not include the underlying address nor the start, which means that a market can be re-initialised with changes to these params. Crucially, start affects `settleFunding()`, which can be DoSed by setting start to be either way in the future or in the past.

POC

Re-initialise the market prior to `manager.syncFunding()` in `test_invariant_accumulator_based_feesOff`, the test will revert with `[FAIL: panic: arithmetic underflow or overflow (0x11)]`.

```
vm.prank(address(GUARDIAN_ADDRESS));
bytes32 createdMktId = vamm.createMarket(
    address(usdc),
    "USD",
```

```
"UNLISTED TOKEN",
/// T0 → Te, 20%,
/// eg 8%, 6%, ---> -2%,
/// 100 * 0.08, 8 ---> 6
200_000e18, /// 0.2, expecting the price 1.1 → 1.3, .2
1_000_000e18,
0,
defaultMktMaturity,
0
);
assertEq(createdMktId, defaultMktId);

// Act: one settle step (via manager)
manager.syncFunding(mkt); // calls vault.settleFunding(...)
```

3. mktId is derived from tokenA, tokenB, maturity and fee. However, fee is mutable via setFeeForMarket(), so once a fee gets updated to a different value, there will be an identity mismatch when attempting to re-derive the mktId from it. Another example is where two markets with the same economic parameters (same underlying + maturity) but different initial fees can be created, then subsequently the same fee to both, resulting in 2 "identical" markets with different market IDs.

Recommendations:

- Ensure that existing markets cannot be re-initialised in SuperRateSwapVAMMSingleton.createMarket(); although this assumes that the vAMM is the only entrypoint; the manager, vault and book have initialisation functions that can be called by authorized entities.

```
if (markets[key].maturity != 0) revert ExistingMarket();
```

- Market ID derivation should omit the fee and include the underlying and start parameters.

```
key = keccak256(abi.encode(tokenA, tokenB, maturity, fee));
key = keccak256(abi.encode(underlying, tokenA, tokenB, start, maturity));
```

- Sanity check the tokenBInitial, start, maturity and fee parameters.

Supernova: Resolved with [PR #111](#).

Zenith: Verified.

[L-14] Hardcoded maximum fee value is incorrect

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [vAMMSingleton.sol#L143](#)

Description:

The fee validation check in `setFeeForMarket()` uses an incorrect constant that allows fees far exceeding the intended maximum of the assumed 50%. Since `10_000` denominator is 100%, $0.5e5 = 50_000$, which would be 500%.

Recommendations:

```
if (newFee > 0.5e5) revert InvalidFee();  
if (newFee > 5_000) revert InvalidFee(); // 50%
```

Supernova: Resolved with [PR #112](#).

Zenith: Verified.

[L-15] Lack of liquidator incentive leads to bad debt accumulation

SEVERITY: Low

IMPACT: Low

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [manager.sol#L512-L530](#)

Description:

The liquidation mechanism in Manager fails to incentivize liquidators when a position is fully insolvent (collateral is zero or insufficient to cover the penalty). The liquidator reward is calculated as a percentage of the user's remaining collateral.

```
function liquidate(bytes32 mktId, address user) external {
    accrueToCollateral(mktId, user);
    Position memory p = positions[mktId][user];
    bool _isSolvent = _checkSolvency(p, mktId, false);
    if (!_isSolvent) revert NotUnderwater();
    uint baseIn = vamm.closeLiquidation(mktId, user, p);
    uint maxDue = (baseIn * 0.05e5) / 1e5;
    if (positions[mktId][user].base > maxDue) {
        positions[mktId][user].base -= maxDue;
        clearingHouse.credit(msg.sender, (maxDue * 0.9e5) / 1e5);
        clearingHouse.credit(address(this), (maxDue * 0.1e5) / 1e5);
    } else {
        maxDue = positions[mktId][user].base;
        positions[mktId][user].base = 0;
        clearingHouse.credit(msg.sender, (maxDue * 0.9e5) / 1e5);
        clearingHouse.credit(address(this), (maxDue * 0.1e5) / 1e5);
    }
    emit PositionLiquidiated(mktId, user, msg.sender, baseIn, (maxDue * 0.9e5) / 1e5);
}
```

If `positions[mktId][user].base` is 0 (or very small), the liquidator receives 0 reward. Rational liquidators will not liquidate these positions.

Recommendations:

Ensure there is always a minimum liquidator reward, potentially paid from a reserve fund if the user is insolvent. Alternatively, require a minimum margin that is sufficient to cover gas costs + incentive even in worst-case scenarios.

Supernova: Acknowledged. We will be running in house liquidators that are more for the purpose of protocol health as opposed to a strict profit maximization incentive.

[L-16] Solvency check division by zero

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L699](#)

Description:

`_checkSolvency` calculation reverts if `p.base` is 0. This prevents clearing/liquidating insolvent positions. This is possible if user have short position and funding is owed, it subtracts from `p.base`. This can drive `p.base` to zero over time.

```
function _checkSolvency(
    Position memory p,
    bytes32 mktId,
    bool isSafeCheck
) internal view returns (bool isSolvent) {
    uint256 price = vamm.getTimeWeightedPricePerMarketCurrent(mktId);
    uint256 ltvForMarket = mktIdToLtv[mktId];
    // Lower the LTV by 10% to reduce liquidation stress, if applicable
    if (isSafeCheck) ltvForMarket = (ltvForMarket * 90) / 100;
    if (p.quote >= 0) {
        // Margin For Long, not allowed
        isSolvent = true;
    } else {
        uint variableLegDebt = (uint(-p.quote) * price) / 1e18;
        > uint health = (variableLegDebt * 1e5) / p.base; // debt // collateral
        isSolvent = health <= ltvForMarket;
    }
}
```

Recommendations:

Return false immediately if `p.base == 0` and `variableLegDebt > 0`.

Supernova: Resolved with [PR#120](#).

Zenith: Verified.

[L-17] Vault insolvency freezes market

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [Vault.sol#L190](#)

Description:

In vault, if the contract's PnL is negative and exceeds its total assets (insolvency), the subtraction `s.totalAssets - _abs(s.vaultPnL)` will revert. Since this logic is executed in `settleFunding` (called by `Manager.syncFunding` in almost every interaction), an insolvent vault will cause the entire market to revert all transactions, freezing it permanently.

```
function settleFunding(
    bytes32 mktId,
    uint256 growthRay
) external onlyRole(MANAGER_ROLE) {
    // ...

    // Handle maturity settlement
    if (s.maturityTs <= block.timestamp && s.vaultPnL != 0) {
        s.totalAssets = s.vaultPnL < 0
        > ? s.totalAssets - _abs(s.vaultPnL) // @audit - this can revert
        : s.totalAssets + uint(s.vaultPnL);
        s.finalVaultPnL = s.vaultPnL;
        s.finalVaultAccrued = s.accruedFunding;
        s.vaultPnL = 0;
    }

    emit FundingSettled(
        mktId,
        s.longPool.accPerNotionalWad,
        s.shortPool.accPerNotionalWad,
        uint(fees)
    );
}
```


Recommendations:

Handle insolvency gracefully via `saturatingSub`.

```
// Taken from Solady's FixedPointMathLib
/// @dev Returns `max(0, x - y)`.
function saturatingSub(uint256 x, uint256 y)
    internal pure returns (uint256 z) {
    /// @solidity memory-safe-assembly
    assembly {
        z := mul(gt(x, y), sub(x, y))
    }
}
```

```
? s.totalAssets - _abs(s.vaultPnL)
? saturatingSub(s.totalAssets, _abs(s.vaultPnL))
```

Supernova: Resolved with [PR #121](#).

Zenith: Verified.

[L-18] Missing deadline check in `openSession`

SEVERITY: Low

IMPACT: Low

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [RateSwapRouter.sol#L292-L332](#)

Description:

The `openSession` function accepts a deadline parameter which is accepted in the signature, but `openSession` never verifies if `block.timestamp ≤ deadline`. While the signature itself attests to the deadline.

Recommendations:

Consider adding the deadline check.

Supernova: Resolved with [PR #117](#).

Zenith: Verified.

4.4 Informational

A total of 16 informational findings were identified.

[I-1] `openSession()` can only be called by stored relayer

SEVERITY: Informational

IMPACT: Informational

STATUS: Acknowledged

LIKELIHOOD: Low

Target:

- [RateSwapRouter.sol#L303](#)

Description:

In the `openSession()` function, the `if (delegate  $\neq$  relayer()) revert InvalidCaller();` check only allows a single relayer address to be used as a delegate, even though `RELAYER_ROLE` can be granted to multiple addresses.

Recommendations:

Replace the single address check with a role-based check using `hasRole()` that is used by the other router functions.

```
function openSession(
    address user,
    address delegate,
    uint256 validAfter,
    uint256 validUntil,
    uint256 flags,
    uint256 deadline,
    bytes calldata signature
) external {
    external hasRole(RELAYER_ROLE) {
        if (delegate  $\neq$  msg.sender) revert InvalidCaller();
        if (delegate  $\neq$  relayer()) revert InvalidCaller();
```

As an optimisation, it seems that storing the relayer is redundant given the `RELAYER_ROLE`.

Consider removing this stored value.

Supernova: Acknowledged

[I-2] Rounding dust from liquidation calculations

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Medium

Target:

- [manager.sol#L512-L529](#)

Description:

Liquidation rewards are computed using integer division in two stages (first to derive `maxDue`, then to split it 90/10 between the liquidator and protocol). This may create dust that is never credited to any account but is fully removed from the liquidated position, leading to a small but systematic loss of value.

Example:

- `baseIn = 24680`
- `maxDue = 24680 * 0.05e5 / 1e5 = 1234`
- `liqShare = (maxDue * 0.9e5) / 1e5 = 1110`
- `protShare = (maxDue * 0.1e5) / 1e5 = 123`
- `liqShare + protShare = 1233 < maxDue = 1234`

As such, 1 wei goes unaccounted for.

Recommendations:

Use subtraction to ensure full accounting.

```
uint256 liquidatorShare = (maxDue * 0.9e5) / 1e5;
uint256 protocolShare = maxDue - liquidatorShare; // remainder to protocol

clearingHouse.credit(msg.sender, liquidatorShare);
clearingHouse.credit(address(this), protocolShare);
```

Supernova: Resolved with [PR# 147](#).

Zenith: Verified.

[I-3] p.base assignment can be refactored out

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [vAMMSingleton.sol#L276-L280](#)

Description:

In `closeLiquidation`, the `p.base` adjustment logic is duplicated across the if/else branches when `p.base < baseIn`. Both branches effectively set `p.base` to 0, but the code structure obscures this equivalence.

Recommendations:

```
if (p.base < baseIn) {
    uint deficit = baseIn - p.base;
    if (clearingHouse.userToAccountCredit(user) >= deficit) {
        manager.rawDebit(user, deficit);
        p.base += deficit;
        p.base -= baseIn;
    } else {
        manager.absorb(mktId, user, deficit);
        p.base = 0;
    }
    p.base = 0;
} else {
    p.base -= baseIn;
}
```

Supernova: Resolved with [PR# 142](#).

Zenith: Verified.

[I-4] ZeroOut condition can be simplified

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [RateSwapVammLib.sol#L136](#)

Description:

The referenced condition can be simplified for clarity since quoteOutTarget is uint256.

Recommendations:

```
if (!(quoteOutTarget > 0)) revert ZeroOut();  
if (quoteOutTarget = 0) revert ZeroOut();
```

Supernova: Resolved with [PR# 141](#).

Zenith: Verified.

[I-5] Break condition can be performed earlier

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [OrderBook.sol#L370-L375](#)

Description:

If the fBase conversion is zero, quoteTraded will have no change, so its check can be shifted above it.

Recommendations:

```
// Break if the conversion results in sub 0
if ((fBase * 1e18) / spotPrice == 0) {
  break;
}

quoteTraded += (fBase * 1e18) / spotPrice;

// Break if the conversion results in sub 0
if ((fBase * 1e18) / spotPrice == 0) {
  break;
}
```

Supernova: Resolved with [PR# 140](#).

Zenith: Verified.

[I-6] getLiquidationAPR() doesn't preview funding accrual

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L593-L607](#)

Description:

The `getLiquidationAPR` function directly accesses position storage without previewing accrued funding, leading to slightly inaccurate liquidation APR calculations.

Recommendations:

- Use `getPosition(mktId, user, true)` instead of directly accessing storage to ensure accrued funding is previewed before calculating liquidation APR.

```
function getLiquidationAPR(  
    bytes32 mktId,  
    address user  
) external view returns (uint256) {  
    uint256 ltvForMarket = mktIdToLtv[mktId];  
    Position memory p = positions[mktId][user];  
    Position memory p = getPosition(mktId, user, true);  
}
```

Supernova: Resolved with [PR# 135](#).

Zenith: Verified.

[I-7] Redundant re-computation of defaultMargin

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L338](#)

Description:

The `debitQuoteMarket` function redundantly recalculates `defaultMargin` with identical parameters to the initial calculation a few lines above. Since neither `mkt` nor `amount` change between these calculations, the recalculation is unnecessary and wastes gas.

Recommendations:

Remove the referenced line.

Supernova: Resolved with [PR# 133](#).

Zenith: Verified.

[I-8] Redundant accrual preview in fetchPosition()

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [manager.sol#L503-L504](#)

Description:

In `fetchPosition`, the third argument to `getPosition` is `true` (`previewAccrued`). Since `accrueToCollateral` is called immediately before, the position in storage is already fully accrued; passing `previewAccrued=true` is redundant.

With `previewAccrued=false`, `getPosition` returns `positions[mktId][user]` as-is, which is already the accrued position.

Recommendations:

Pass `false` for the third parameter, since the position has already been accrued:

```
function fetchPosition(  
    bytes32 mktId,  
    address user  
) external returns (Position memory p) {  
    accrueToCollateral(mktId, user);  
    return getPosition(mktId, user, true);  
    return getPosition(mktId, user, false);  
}
```

Supernova: Resolved with [PR #129](#).

Zenith: Verified.

[I-9] ClearingHouse unit mismatch

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [ClearingHouse.sol#L182-L192](#)
- [ClearingHouse.sol#L194-L206](#)

Description:

ClearingHouse.deposit expects 6-decimal USDC amounts, while withdraw expects 18-decimal internal credit units.

```
function _deposit(bool transferUnderlying, uint256 _amount, address _user)
    internal {
    ClearingHouseStorage storage $ = _getClearingHouseStorage();
    if (transferUnderlying) usdc.transferFrom(_user, address(this), _amount);
    emit AccountCredited(true, _user, _amount * 1e12);
    if ($.userToGasBalance[_user] == 0) {
        _amount -= $.minimumGasForNewAccount / 1e12;
        \($.userToGasBalance[_user] += \).minimumGasForNewAccount;
        emit AccountGasCredited(_user, $.minimumGasForNewAccount);
    }
    > $.userToAccountCredit[_user] += _amount * 1e12;
    }

function _withdraw(uint256 _amount, address _user) internal {
    ClearingHouseStorage storage $ = _getClearingHouseStorage();
    uint gasCredited;
    > $.userToAccountCredit[_user] -= _amount;
    if ($.userToAccountCredit[_user] == 0) {
        gasCredited = $.userToGasBalance[_user];
        _amount += gasCredited;
        emit AccountGasDebited(true, _user, gasCredited);
        $.userToGasBalance[_user] = 0;
    }
    usdc.transfer(_user, _amount / 1e12);
    emit AccountDebited(true, _user, _amount);
    }
```

Recommendations:

Standardize interfaces to either use 18 or 6 decimals or document clearly.

Supernova: Resolved with [PR #127](#)

Zenith: Verified. Documentation clarifying the decimal differences has been added.

[I-10] Typographical Errors

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [RateSwapRouter.sol](#)
- [OrderBook.sol](#)
- [manager.sol](#)
- [OrderBookBase.sol](#)
- [Vault.sol](#)
- [vAMMSingleton.sol](#)
- [ClearingHouse.sol](#)

Description:

Multiple typographical errors were found throughout the codebase in comments, documentation strings, and error names. The errors include misspellings in function documentation, comment headers, and error revert names.

Recommendations:

RateSwapRouter.sol

```
/// typing given the size constraints surrounding the VAMM signleton
/// typing given the size constraints surrounding the VAMM singleton

/// @dev Indended to be a proxy,
need to price book in terms of non-decaying pricing
/// @dev Intended to be a proxy,
need to price book in terms of non-decaying pricing

/// @dev Indended deployment setup similar to router on Uni/Sushi et al
/// @dev Intended deployment setup similar to router on Uni/Sushi et al
```

```
// keccak256(bytes("PlaceAsk(bytes32 mktId,address user,uint256 limitRay,
uint256 size,uint256 deadline,
uint256 nonce)"))
// keccak256(bytes("PlaceAsk(bytes32 mktId,address user,uint256 limitRay,
uint256 size,uint256 deadline,
uint256 nonce)"))

// keccak256(bytes("PlaceBid(bytes32 mktId,address user,uint256 limitRay,
uint256 size,uint256 deadline,
uint256 nonce)"))
// keccak256(bytes("PlaceBid(bytes32 mktId,address user,uint256 limitRay,
uint256 size,uint256 deadline,
uint256 nonce)"))

// keccak256(bytes("CreditGas(address user,
uint256 amount,uint256 deadline,uint256 nonce)"))
// keccak256(bytes("CreditGas(address user,
uint256 amount,uint256 deadline,uint256 nonce)"))

// keccak256(bytes("OpenSession(address user,
address delegate,uint256 validAfter,uint256 validUntil,
uint256 flags,uint256 deadline,uint256 nonce)"))
// keccak256(bytes("OpenSession(address user,
address delegate,uint256 validAfter,uint256 validUntil,
uint256 flags,uint256 deadline,uint256 nonce)"))

// keccak256(bytes("DepositToVault(bytes32 mktId,
address user,uint256 amount,uint256 deadline,
uint256 nonce)"))
// keccak256(bytes("DepositToVault(bytes32 mktId,
address user,uint256 amount,uint256 deadline,
uint256 nonce)"))

// keccak256(bytes("WithdrawFromVault(bytes32 mktId,
address user,uint256 shares,uint256 deadline,
uint256 nonce)"))
// keccak256(bytes("WithdrawFromVault(bytes32 mktId,
address user,uint256 shares,uint256 deadline,
uint256 nonce)"))
```

```
/// @notice Post deployment setter for clearingHouse
/// @notice Post deployment setter for clearingHouse

/// @notice EIP-712 Domain Separator for signature support
/// @return The bytes32 domain separator for this contract
/// @notice EIP-712 Domain Separator for signature support
/// @return The bytes32 domain separator for this contract
```

OrderBook.sol

```
/// @notice Protected function callable via router,
places a limit order on the Short side
/// @notice Protected function callable via router,
places a limit order on the Short side
```

manager.sol

```
* @notice function to set the initial state w/n the context of the proxy
* @notice function to set the initial state w/n the context of the proxy

* @param mktId The bytes32 identifier for the market being deposited to
* @param mktId The bytes32 identifier for the market being deposited to

* @param user The address of the user who is withdrawing
* @param user The address of the user who is withdrawing

* @param user The users whose position is being accrued to
* @param user The users whose position is being accrued to
```


OrderBookBase.sol

```
// Internal Book Queires //  
// Internal Book Queries //  
  
/// @notice Given a book will return  
the price (Rate Ray) for the top bid of the book  
/// @notice Given a book will return  
the price (Rate Ray) for the top bid of the book  
  
/// @notice Given a book will return  
the price (Rate Ray) for the lowest ask of the book  
/// @notice Given a book will  
return the price (Rate Ray) for the lowest ask of the book
```

Vault.sol

```
if (isSubscriptionClosed[mktId]) revert SubscriptionClosed();  
if (isSubscriptionClosed[mktId]) revert SubscriptionClosed();  
  
error SubscriptionClosed();  
error SubscriptionClosed();
```

vAMMSingleton.sol

```
/// @param tokenA String representing the base token for a mkt  
/// @param tokenA String representing the base token for a mkt
```

ClearingHouse.sol

```
// Gassless Relayed //  
// Gasless Relayed //
```

```
/// @notice EIP-712 Domain Separator for signature support
/// @return The bytes32 domain separator for this contract
/// @notice EIP-712 Domain Separator for signature support
/// @return The bytes32 domain separator for this contract
```

Supernova: Resolved with [PR #110](#), additionally includes other typos that were not mentioned.

Zenith: Verified.

[I-11] Missing structs for typehashes

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [RateSwapRouter.sol#L54-L96](#)

Description:

RateSwapRouter defines EIP-712 typehash constants, but does not define corresponding struct types. While the current implementation works correctly (using `abi.encode()` directly with pre-computed typehashes and values), defining structs would be helpful when generating signatures off-chain.

Recommendations:

Add struct definitions matching each typehash, some examples below.

Note: There is a name mismatch between `limitRay` and `limitRateRay` used in the `placeBid()` and `placeAsk()` functions. Consider renaming one to the other for consistency.

```
struct PlaceBid {
    bytes32 mktId;
    address user;
    uint256 limitRay; // Note: matches typehash, not function parameter name
    uint256 size;
    uint256 deadline;
    uint256 nonce;
}

struct Swap {
    bytes32 mktId;
    bool onlyUseBook;
    bool onlyUseAmm;
    bool isLong;
    uint256 amountIn;
    uint256 minAmountOut;
    address user;
```

```
uint256 deadline;  
uint256 nonce;  
}
```

Supernova: Resolved name mismatch with [PR #110](#). Acknowledged the struct definition inclusions.

Zenith: Verified.

[I-12] USDC hardcoded value may have to be changed for MegaETH

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [ClearingHouse.sol#L48-L49](#)

Description:

It is unknown if MegaETH mainnet's USDC token address will correspond to Arbitrum's USDC address. If they are different, then the value will have to be changed prior to deployment.

Recommendations:

Consider making it immutable instead of a constant, and pass it as a constructor param.

Supernova: Resolved with [PR #113](#).

Zenith: Verified.

[I-13] Struct memory copying is by reference

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [RateSwapVammLib.sol#L93](#)
- [RateSwapVammLib.sol#L115](#)
- [vAMMSingleton.sol#L210](#)
- [vAMMLib.t.sol#L126-L131](#)

Description:

Assigning a struct from one memory variable to another creates a reference to the same data in memory. Modifying one affects the other. Hence, modifications to `ns` will also apply to `s`, making some of the `vAMMLib` tests false positives.

Note that the `baseDiff` calculation within `SuperRateSwapVAMMSingleton.swap()` := `int(m.base) - int(baseStart)`; would thus mean that `m.base` is the modified value, ie. `ns.base`, which seems to be the desired behaviour.

Recommendations:

For clarity, either remove `ns` entirely and make it explicit that `s` is being mutated, or if really needed, explicitly clone `s` into `ns`. Based on the implementation and use-case, the former is preferred, as it will provide some gas optimizations as well (`ns` can be removed entirely within `SuperRateSwapVAMMSingleton.swap()`), for instance.

An example of the updated library functions is given below.

```
function swapBaseIn(State memory s, uint256 baseIn)
    internal pure returns (uint256 quoteOut, uint256 fee) {
    if (!(baseIn > 0)) revert ZeroIn();
    uint256 newBase = s.base + baseIn;
    uint256 newQuote = s.k / newBase; // y' = k/(x+dx)
    uint256 rawOut = s.quote - newQuote; // Δy
    fee = (rawOut * s.feeBps) / 10_000; // fee on output
    quoteOut = rawOut - fee;
    s.base = newBase;
```

```
s.quote = s.quote - rawOut;
// ns.quote = s.quote - quoteOut;
}

function swapQuoteIn(State memory s, uint256 quoteIn)
    internal pure returns (uint256 baseOut, uint256 fee) {
    if (!(quoteIn > 0)) revert ZeroIn();
    uint256 newQuote = s.quote + quoteIn;
    uint256 newBase = s.k / newQuote; // x' = k/(y+dy)
    uint256 rawOut = s.base - newBase; // Δx
    fee = (rawOut * s.feeBps) / 10_000; // fee on output
    baseOut = rawOut - fee;

    s.quote = newQuote;
    s.base = s.base - rawOut;
    // ns.base = s.base - baseOut; // fee stays inside reserves
}
```

Supernova: Resolved with [PR #114](#).

Zenith: Verified.

[I-14] Unsafe castings

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [RateSwapRouter.sol#L328-L330](#)
- [RateSwapRouter.sol#L86-L88](#)

Description:

There are no checks to ensure the values fit within the target types before casting, unlike explicit bounds checking that would revert on overflow. For example, `flags = type(uint96).max + 1` will be equivalent to 0.

Recommendations:

Change the types of `validAfter` and `validUntil` to `uint64`, `flags` to `uint96` so that casting becomes redundant.

```
// keccak256(bytes("OpenSession(address user,address delegate,uint256
    validAfter,uint256 validUntil,
    uint256 flags,uint256 deadline,uint256 nonce)"))
// keccak256(bytes("OpenSession(address user,address delegate,uint64
    validAfter,uint64 validUntil,
    uint96 flags,uint256 deadline,uint256 nonce)"))
0x325edc56b287ed5a6e097670944a2535a1b229fe1d10f8d67229be76be53718c
0x2e7ae714b0c19797635df632f9f22b3f10954c0e4ef6f49758de10c3c23394b0

function openSession(
    address user,
    address delegate,
    uint256 validAfter,
    uint256 validUntil,
    uint256 flags,
    uint64 validAfter,
```



```
uint64 validUntil,  
uint96 flags,  
uint256 deadline,  
bytes calldata signature  
) external {  
    // ...  
    $.session[user][delegate].validAfter = uint64(validAfter);  
    $.session[user][delegate].validUntil = uint64(validUntil);  
    $.session[user][delegate].flags = uint96(flags);  
    $.session[user][delegate].validAfter = validAfter;  
    $.session[user][delegate].validUntil = validUntil;  
    $.session[user][delegate].flags = flags;  
}
```

Supernova: Resolved with [PR #115](#).

Zenith: Verified.

[I-15] Incorrect natspec / comments

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [vAMMSingleton.sol#L220](#)
- [OrderBookBase.sol#L147](#)
- [manager.sol#L533](#)
- [manager.sol#L699](#)

Description:

```
/// @return amountIn The amount of base needed to close quote exposure
/// @return amountIn The base / quote amount needed to close short / long
    exposure respectively
function closeOpenPosition(
```

Maybe add a short @dev comment or an additional sentence on how only short exposure can be liquidated => thus base amount only in closeLiquidation().

```
// Taker is buying selling → maker is the asker (going short)
// Taker is buying quote → maker is the asker (going short)
```

```
Alternative Liquidation function,
where the user will take over the position
Alternative Liquidation function,
where the caller will take over the position
```

```
uint health = (variableLegDebt * 1e5) / p.base; // debt // collateral  
uint health = (variableLegDebt * 1e5) / p.base; // debt / collateral
```

Recommendations:

See description for remediation details.

Supernova: Resolved with [PR #116](#).

Zenith: Verified.

[I-16] Deposit can revert when vault headroom is zero

SEVERITY: Informational

IMPACT: Informational

STATUS: Resolved

LIKELIHOOD: Low

Target:

- [Vault.sol#L340](#)

Description:

If the vault becomes insolvent ($\text{totalAssets} + \text{vaultPnL} < 0$), `_vaultHeadroomLessAccrued` returns 0. In `depositToVault`, the share price calculation divides by this value:

```
function depositToVault(
    bytes32 mktId,
    address user,
    uint256 amountAsset
) external onlyRole(ROUTER_ROLE) returns (uint256 shares) {
    Mkt storage s = mkt[mktId];
    if (isSubscriptionClosed[mktId]) revert SubscriptionClosed();
    if (s.maturityTs > block.timestamp) manager.syncFunding(mktId);
    if (block.timestamp > s.maturityTs) revert SubscriptionClosed();
    if (amountAsset + s.totalAssets > s.vaultCap) revert VaultCapExceeded();
    clearingHouse.debit(user, amountAsset);
    if (s.totalShares != 0) {
        > shares = (amountAsset * s.totalShares)
            / uint(_vaultHeadroomLessAccrued(mktId));
    } else {
        shares = amountAsset;
    }
    balance[mktId][user] += shares;
    s.totalShares += shares;
    s.totalAssets += amountAsset;
    emit DepositToVaultMarket(mktId, user, amountAsset, shares);
}
```

This causes a division-by-zero revert. While this may be intentional to prevent deposits into an insolvent vault, the error itself may not clearly communicate the actual reason for the revert.

Recommendations:

Handle the 0 headroom case gracefully.

Supernova: Resolved with [PR #119](#).

Zenith: Verified.